to decode and verify a JWT using those keys. In the context of a web slice test, we can provide a mock `ReactiveJwtDecoder` bean so that Spring Security skips the interaction with Keycloak (which will be validated by full integration tests).

Listing 12.20 Verifying security policies at the web layer with slice tests

```
@WebFluxTest(OrderController.class)
@Import(SecurityConfig.class)                                    ❶
class OrderControllerWebFluxTests {

  @Autowired
  WebTestClient webClient;

  @MockBean
  OrderService orderService;

  @MockBean
  ReactiveJwtDecoder reactiveJwtDecoder;                         ❷

  @Test
  void whenBookNotAvailableThenRejectOrder() {
    var orderRequest = new OrderRequest("1234567890", 3);
    var expectedOrder = OrderService.buildRejectedOrder(
     orderRequest.isbn(), orderRequest.quantity());
    given(orderService.submitOrder(
     orderRequest.isbn(), orderRequest.quantity()))
        .willReturn(Mono.just(expectedOrder));

    webClient
      .mutateWith(SecurityMockServerConfigurers
        .mockJwt()                                               ❸
        .authorities(new SimpleGrantedAuthority("ROLE_customer")))
      .post()
      .uri("/orders/")
      .bodyValue(orderRequest)
      .exchange()
      .expectStatus().is2xxSuccessful()
      .expectBody(Order.class).value(actualOrder -> {
        assertThat(actualOrder).isNotNull();
        assertThat(actualOrder.status()).isEqualTo(OrderStatus.REJECTED);
      });
   }
}
```

❶ Imports the application security configuration